



ENN

LDA

$classes = np.unique(y)$

⇒ 檢查 y 有多少不同的類別

⇒ $y = [0, 0, 1, 1, 2]$

⇒ $classes = [0, 1, 2]$

$assert len(classes) == 2$

⇒ 判定僅適用二元分類

$c_1, c_2 = classes[0], classes[1]$

⇒ 既然已確認 $classes$ 中只有 "0" "1" 兩種元素

$classes[0] = 1$ 會是第 1 類

$classes[1] = 2$ 會是第 2 類

指在 $classes$ 中的位置 ex. $classes = [1, 2]$

$x_1, x_2 = X[y == c_1], X[y == c_2]$

⇒ x_1, x_2 是矩陣

⇒ $y == c_1$ 會標記一個布林序列，標記哪些樣本屬於 class c_1 .

ie. $y = [1, 1, 2, 2, 1]$, $c_1 = 1, c_2 = 2$

$classes = [1, 2]$

$c_1 = classes[0] = 1, c_2 = classes[1] = 2$

$x_1 = X[1, 1, 2, 2, 1] = X[y == 1]$

$= [T, T, F, F, T] \Rightarrow$ 只保留屬於 c_1 的樣本

$x_2 = [F, F, T, T, F] \Rightarrow$ 只保留屬於 c_2 的樣本

$m_1, m_2 = x_1.mean(axis=0), x_2.mean(axis=0)$

⇒ 對 x_1, x_2 每一欄 $a[i]$ 作均值

$S_i = np.cov(x_1, \text{rowvar=False, bias=False}) \rightarrow$ 看各自分佈

共變異矩陣 每一行是特徵 無偏估計

$$S_2 = \text{np.cov}(X_2, \text{rowVar} = \text{false}, \text{bias} = \text{false})$$

$$n_1, n_2 = \text{len}(X_1), \text{len}(X_2)$$

X_1, X_2 的樣本數

$$S_W = (n_1 - 1) S_1 + (n_2 - 1) S_2 / (n_1 + n_2 - 2) \rightarrow \text{合併的總體分布}$$

共變異矩陣

$$\Rightarrow S_W = \frac{(n_1 - 1) S_1 + (n_2 - 1) S_2}{n_1 + n_2 - 2}$$

$$\text{ridge} = 1e-6$$

$$S_W + \text{ridge} * \text{np.eye}(S_W.\text{shape}[0]) = I_{S_W}$$

$\Rightarrow S_W$ 是 $d \times d$ 的方陣

$\text{np.eye}(S_W.\text{shape}[0])$ 生成單位矩陣 大小跟 S_W 一樣

$\Rightarrow \text{ridge} = 1e-6$ 稜背項 用於數值穩定化

$$S_W = S_W + \text{ridge} * \text{np.eye}(\text{shape}(S_W)[0])$$

$$= S_W + \text{ridge} * I$$

把共變數矩陣 S_W 的對角線元素每個都加上跟 ridge 的 ridge term

$\Rightarrow$ 可以避免矩陣不可逆 $S_W^{-1}(m_1 - m_2)$

$$w = \text{np.linalg.solve}(S_W, (m_1 - m_2))$$

$$= \sum^T (m_1 - m_2) \sim \text{np.linalg.inv}(S_W) @ (m_1 - m_2)$$

* 為何 solve 比 inv 強

$\Rightarrow \text{inv}$ 會真的把矩陣 S_W 反轉, 易有浮點誤差

$$\Rightarrow \text{np.linalg.solve}(S_W, (m_1 - m_2))$$

會直接解 $S_W \cdot w = (m_1 - m_2)$

$$\Rightarrow w = S_W^{-1}(m_1 - m_2)$$

$$p_1, p_2 = n_1 / (n_1 + n_2), n_2 / (n_1 + n_2)$$

$$b = -0.5 \cdot 2 \cdot \text{dot}(m_1 + m_2) + 2p \cdot \log(p_1 / p_2)$$

$$\Rightarrow D(x) = W^T x + b$$

$$W = (m_1 - m_2)^T \Sigma^{-1} \quad b = -0.5 (m_1 - m_2)^T \Sigma^{-1} (m_1 + m_2) - \ln\left(\frac{C_1 P_1}{C_2 P_2}\right)$$

$$P_{11} = n_1 / n_1 + n_2 \Rightarrow P_{12} = n_2 / n_1 + n_2$$

$$= P_1 \qquad \qquad \qquad = P_2$$

$$\Rightarrow \text{假設 } C_1 / C_2 = 1$$

```
def LDA_predict(x, W, b, classes)
```

$$\text{Score} = x @ W + b$$

$$C_1, C_2 = \text{classes}$$

$$y_hat = np.where(\text{Score} \geq 0.0, C_1, C_2)$$

```
return y_hat
```

```
def accuracy(y_true, y_pred)
```

```
return np.mean(y_true == y_pred)
```

$\Rightarrow$ 若 $y_{true} = y_{pred}$ 則輸出 true, 反之則輸出 false 為一布林陣列

$\Rightarrow$ 對陣列取 mean 即為 accuracy

```
def rm-binary(x_all, y_all, classes, feat_idx, off_name, f=None)
```

$$C_1, C_2 = \text{class}$$

$$\text{mask} = np.isin(y_all, [C_1, C_2])$$

$\Rightarrow$ 把不是 C_1, C_2 的濾掉

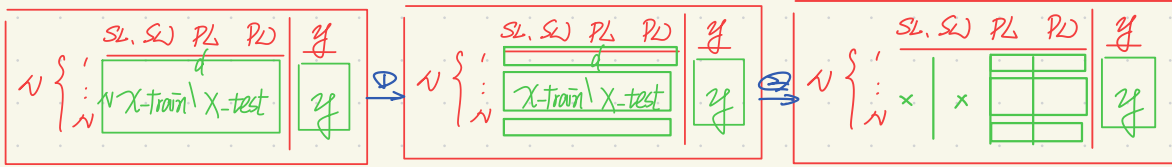
$$x = x_all[\text{mask}][:, \text{feat_idx}]$$

$$\textcircled{1} x_all[\text{mask}] = x_all[0, 1, 1] \text{ 用布林遮罩排列}$$

$\Rightarrow (x_all, d_all)$ 變成 (x_sub, d_all) ; $x_sub = \text{mask} \cdot x_all$ True 的數

* mask 是布林陣列, 每個位置是 T/F

② feat_idx: 通常是整數索引列表, $[2, 3]$ 代表 P_L, P_D
結果形狀會變為 (N_{sub}, d_{sub}) : $d_{sub} = len(feats_idx)$



⇒ 先選想要的樣本, 再選想要的特徵

$y_all[mask]$

只保留 G_1, G_2 , 為 $(N_{sub}, 1)$

$foldA_full, foldB_full = stratified_two_fold(y_all, seed=42)$

$sub_idx = mp.where(mask)[0]$

⇒ 只有 G_1, G_2 的 idx , 只有對應 G_1, G_2 的位置

$A = mp.intersect1d(sub_idx, foldA_full)$

$B = mp.intersect1d(sub_idx, foldB_full)$

會把全集的 sub_idx 對應到 sub_idx 子集

取交集得同時屬於 sub_idx 子集且屬於 $foldA$ 的原始索引

① $foldA_full$ 是將全資料所有類別平均平分

i.e, 150 筆 $\Rightarrow$ 類 (G_1, G_2, G_3) 各 50

⇒ 取完交集後剩 $G_1=50, G_2=50$

$remap = \{old: i \text{ for } i, old \text{ in enumerate}(sub_idx)\}$

⇒ sub_idx 子集樣本在全集的索引

$enumerate(sub_idx) = \text{逐一枚舉}(i, old)$

$remap$ 把原編號 old 對應到新編號 i

X 的索引 $(N_{sub}) \Rightarrow A, B$ 的索引 (N_{all})

$sub_idx = [0, 1, 2, 2, 8]$

$\downarrow$
 $= [0, 1, 2, 3, 4]$

$A = \text{np.array}([\text{remap}[i] \text{ for } i \text{ in } A])$

$B = \text{np.array}([\text{remap}[i] \text{ for } i \text{ in } B])$

i.e., $\text{sub_idx} = [0, 1, 2, 6, 7, 8]$

$\text{remap} = [0, 1, 2, 3, 4, 5]$

$A = [0, 2, 6, 8] \rightarrow [0, 1, 3, 5]$

$B = [1, 7] \rightarrow [1, 4]$

$X_{\text{tr}}, y_{\text{tr}} = X[A], y[A]$ 訓練

$X_{\text{te}}, y_{\text{te}} = X[B], y[B]$ 測試

$X_{\text{tr,z}}, m, s = \text{Standardize-fit}(X_{\text{tr}})$

$X_{\text{te,z}} = \text{Standardize-apply}(X_{\text{te}}, m, s)$

⇒ 使用訓練集的 m, s , 因測試集不含有 m, s , 只能用訓練集的

if clf_name in ("knn", "svm")

$y_{\text{hat}} = \text{knn-predict}(X_{\text{tr,z}}, y_{\text{tr}}, X_{\text{te,z}}, k=k)$

else #LDA

$w, b, \text{classes} = \text{LDA-binary}(X_{\text{tr,z}}, y_{\text{tr}})$

$y_{\text{hat}} = \text{LDA-predict}(X_{\text{te,z}}, w, b, \text{classes_used})$

$\text{acc1} = \text{accuracy}(y_{\text{te}}, y_{\text{hat1}})$

重複 $X_{\text{tr}}, y_{\text{tr}} = X[B], y[B]$

$X_{\text{te}}, y_{\text{te}} = X[A], y[A]$

$\text{acc2} = \text{accuracy}(y_{\text{te}}, y_{\text{hat2}})$

return (acc1+acc2)/2.0